

ICS 214 Lab 2

Objectives:

1. Make use of the debugger and some of its basic features.
2. Differentiate between unscoped enumerations (enums) and scoped enumerations (enum classes).
3. Demonstrate the use of `static_cast`.
4. Demonstrate the use of forward declarations.
5. Demonstrate how to create a multi file project.
6. Demonstrate the use of a header file with header guards

This lab introduces a few new concepts:

Casting (explicit type conversion)

Casting allows us to convert a value from one data type to another (eg: int to double, double to int). In this lab you will need to cast an int as a Weekday. What's a Weekday? It's a custom type that you will be creating using a scoped enumeration. There are several different methods of casting: the old way (C-style casting), and the new C++ way (`static_cast`). We want to understand both, but we will be using the newer style because it is 'type safe'. If you have not yet seen it, have a read through [LearnCpp section 10.6](#) for a description of how (and why) to cast with `static_cast`.

Unscoped enumerations and scoped enumerations.

Unscoped enumerations types allow us to create our own simple data type which is simply a set of constants, each of which has an integer representation. Scoped enumerations are a type-safe (improved) version, so we will be using these. Read through [LearnCpp section 13.6](#) to understand how they work and the differences between them.

The `std::string` class:

C doesn't have a string data type the way other languages do. It stores strings as arrays of characters terminate by a null character. C++ adds the `std::string` class which makes strings friendlier and much easier to use. For this lab you'll use `std::string` (a class in the standard library). The code below should give you the information you need to know and demonstrate its use within the context of this lab:

```
#include <string>
std::string s;           // instantiate a std::string
s = "hello there";      // assign it
std::cout << s;         // print it out;
```

Switch statement:

Switch statements in C++ have the same format as they do in Java. A key difference is that you can't switch on a string. A switch statement only works with integral types. Integral types are based on integers (eg: chars, enums) and can be converted to an int. For a look at switch statement format see [LearnCpp section 8.5 \(Switch statement basics\)](#).

Step 1: Project setup and reusing functionality from Lab 1

Create a new empty project called Lab2. Add a new file to the solution (right click on "Source Files | Add | New Item") and name it TextInput.cpp.

From Lab 1, copy the function: [getIntFromUserInRange\(\)](#) and any functions that it needs to work correctly into TextInput.cpp.

Create a header file (TextInput.h) to pair with TextInput.cpp. Remove #pragma once (if VS2022 automatically puts it at the top of the file). Write the standard C++ header guards for this header file. Match the identifier that you #define for this to the name of the header file. (eg: for a file named Math.h, you would use #define MATH_H). Fill in the header file with forward declarations that will allow other files to use functions in the associated TextInput.cpp.

Step 2: the problem at hand

[Read through all of Step 2 before starting to code!](#)

Add a new file called "main.cpp" and #include "TextInput.h" in your main.cpp. This will allow you to use [getIntFromUserInRange\(\)](#) for validating input for the new code we are writing.

Create a main function and write a solution for the problem detailed on the next page.

As part of your solution:

- create and use a new type called Weekday (using a **scoped enumeration**),
- separate the functionality of your program into the following functions:

```
int promptForYear();
    // prompt the user for the year and return the result
int promptForMonth();
    // prompt the user for the month and return the result
int promptForDayOfMonth();
    // prompt the user for the day of the month (0-31) and return the result
Weekday calculateWeekday(int month, int dayOfMonth, int year);
    // use Zeller's congruence algorithm to calculate and return a Weekday
std::string getWeekdayName(Weekday day);
// There isn't a straightforward way of printing out the text representation of an
// enumerated type, so we'll write our own
```

```
// Given a Weekday, return a string to represent its name.
// You can do this in several ways:
//     1. multiple if/else if statements
//     2. a switch statement
//     3. create an array of string literals, and index the array by
//         doing a static_cast on the Weekday to get its int representation.
// The third option is the best (most elegant/extensible).
// To do this, you need to know how to create an array of string objects (see
// below):
// eg: std::string myArray[] = { "a", "b", "c"};.
```

- Use meaningful variable names! This lab uses a formula in which letters are used to represent things like: the month, the year, etc... Don't just use the letters in the formula (see **3.33 on the next page) as variable names in your program. Why? Because they mean nothing to the average programmer reading your code. The formula used below makes use of letters that represent different part of an equation, but in a week you'll have no idea what h, q, m, k, and j refer to. Replace them with meaningful variable names like dayOfMonth, yearOfCentury. It's going to be a lot easier for both you and anyone else to understand what's going on in your code!
- For input validation, use these ranges: year[0-9999], day[1-31], month[1-12]. Simply validate [0-31] for the day of month - disregard the fact that some months have 28, 29, 30, or 31 days. We don't need to make it overly complex.
- Use the main function to call each the functions listed above as you're writing the functionality described below:

****3.33** (Science: day of the week) Zeller's congruence is an algorithm developed by Christian Zeller to calculate the day of the week. The formula is

$$h = \left(q + \frac{26(m + 1)}{10} + k + \frac{k}{4} + \frac{j}{4} + 5j \right) \% 7$$

where

- **h** is the day of the week (0: Saturday, 1: Sunday, 2: Monday, 3: Tuesday, 4: Wednesday, 5: Thursday, 6: Friday).
- **q** is the day of the month.
- **m** is the month (3: March, 4: April, ..., 12: December). January and February are counted as months 13 and 14 of the previous year.
- **j** is the century (i.e., $\frac{year}{100}$).
- **k** is the year of the century (i.e., $year \% 100$).

Note that the division in the formula performs an integer division. Write a program that prompts the user to enter a year, month, and day of the month, and displays the name of the day of the week. Here are some sample runs:



```
Enter year: (e.g., 2012): 2015 ↵Enter
Enter month: 1-12: 1 ↵Enter
Enter the day of the month: 1-31: 25 ↵Enter
Day of the week is Sunday
```



```
Enter year: (e.g., 2012): 2012 ↵Enter
Enter month: 1-12: 5 ↵Enter
Enter the day of the month: 1-31: 12 ↵Enter
Day of the week is Saturday
```

(Hint: January and February are counted as 13 and 14 in the formula, so you need to convert the user input 1 to 13 and 2 to 14 for the month and change the year to the previous year.)

***Note:** 3 important things that most students miss with this assignment:

1. The month input to `calculateWeekday()` will be between 1 and 12... the month used in the formula is required to be between 3 and 14. It is probably easiest to calculate a "formulaMonth" variable (a conversion between the two) for use in the formula.
2. If the month was Jan or Feb, the year used in `calculateWeekday()` will need to be decremented by 1.
3. If the year input to your function is 1985, the century will be 19, and the year of the century will be 85.

You should test your program on several dates that you know the weekday of to verify it works. Test some dates at the beginning, middle, and end of the year to prove to yourself it works. Definitely test the 2 examples shown in the description (**3.33) above.

Debugging Tip: Following the details in the instructions are important here. Even if you are amazing at details and brilliant at matching open/close parenthesis, it is difficult to get the correct answers on your first try. I am expecting that you will need to set breakpoints and look at the values that your program is generating to get this working!

To ensure calculations are what expect, I recommend the following:

Take the first example from above (2015, 1, 25), and use the formula to calculate the result on paper to verify that works as expected (it should produce 1 or "Sunday" as a result.) Feel free to use a calculator, but remember that integer division is being used - meaning a calculation like $14/4 == 3$. If you can't make the numbers work on paper - you'll need to figure out what you're doing wrong there first.

Once you get the correct result (1 or Sunday) on paper, you can breakpoint the calculation in code. Compare the paper data you expect to see against the data that you are seeing as the code runs (via the debugger) to determine what is not measuring up.

This can save you a lot of time guessing what might be wrong. Errors are usually due to a detail in the 3 tips highlighted above. Expect to need to use the debugger in this lab for this purpose. (Unless of course you get the correct answers on both text cases on the first try!)

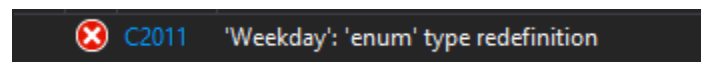
Step 3: move weekday functionality to a separate .cpp file

Add a new file to your project (Weekday.cpp) and move your function definitions there (except for main()) along with any scoped enumerations (that are specific to Weekday functionality). Get your program compiling and running again. To do this - you will have to add new forward declarations (for functions defined in Weekday.cpp) in the main.cpp file. If you have a scoped enumeration (called Weekday) that is used in main(), you will also need to forward declare it in main.cpp using the syntax: `enum class Weekday;`

Step 4: write a matching header file & header guard

Add a matching header file for the code in Weekday.cpp. Comment out the “#pragma once” that the IDE automatically adds. Move all your forward declarations in main.cpp to the header file. Move the Weekday enum class to the header file as well. Get your program compiling and running again. To do this, you will need to #include the new header file in main.cpp so it is aware of the functions it can use. You’ll also need to #include the header file in Weekday.cpp since the functions it contains need to use the Weekday enumerated type.

To illustrate why we need a header guard, #include “Weekday.h” twice in main.cpp. Try compiling. The compiler should present an error:



Recall part 1 of the **One Definition Rule (ODR)**:

1. *Within a given file, a function, variable, **type**, or template can only have one definition.*

The code would have compiled fine if the .h file only contained declarations since the ODR doesn't apply to declarations. However, as soon as we have a type defined in the header (which is common), it could potentially break part 1 of the ODR.

To prevent this from happening, add in a header guard to your .h file.

Technically you *can* use `#pragma once` to accomplish the same thing, but to ensure you to understand how header guards and `#define/#ifndef/#endif` statements work – let’s go through the process of using them before we resort to the easier (but non-standard) way.

The header guards will prevent the Weekday enumeration (a type definition) from being included in the main.cpp compilation twice in case the header file ever gets included more than once (either directly or transitively) in any given .cpp file.

Step 5: Add comments

Add a comment for each function describing what it does **in your header files**. It is standard practice to put leading function comments in your .h file (instead of the .cpp file). The .h file is where a programmer would first go when learning to use a new library. Your documentation is successful if another programmer doesn’t need to examine your .cpp file to figure out how the function should be used.

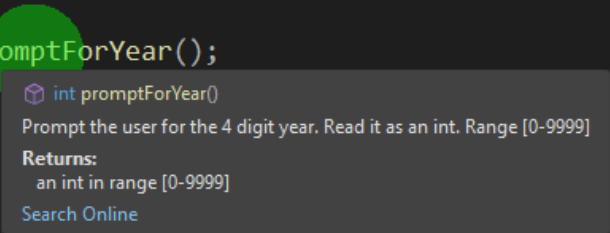
Two useful tips about comments in Visual Studio 2022:

1. VS2022 will automatically pop up a comment template (that you can fill in) for your function if you enter “`///`” on an empty line above a function or function prototype.

```
8  /// <summary>
9  /// Prompt the user for the 4 digit year. Read it as an int. Range [0-9999]
10 /// </summary>
11 /// <returns>an int in range [0-9999]</returns>
12 int promptForYear();
```

2. If you roll over a function call in your code (in VS2022), it will display the comment (that you entered at the top of your function declaration) onscreen so that anyone using it (including you) has access to the description without the need to open the .h file.

```
15 int main()
16 {
17     int year = promptForYear();
18
19
20
21
22
```



Code Review

When you are finished this assignment, find a friend who is also finished. Using the checklist (D2L | Extras | LabCodeReview), conduct a code review on each other's code **before** giving a demo of your assignment for marks.

Marking [out of 10 points]

- Step 1: [1 pts] Setup of TextInput cpp & h files with header guards
- Step 2: [6 pts] (2 points per test case)
 - Test 1: 2015, 1, 25 == Sunday
 - Test 2: 2012, 5, 12 == Saturday
 - Test 3: Today's date works correctly
- Step 3: [1 pt]
- Step 4: [1 pt]
- Add a comment before every function declaration (header files) [1 pt]

Things I am looking for when I'm checking your program:

- Does it work as expected?
- Variables: Have good variable & const names been used?
- No global variables
- Are there comments to explain anything non-trivial or not obvious?
- Does each function declaration have a one line comment explaining what it does?
- Does the program work with valid input?

Due dates:

You have one week to complete this lab. It will be due a week from your regular lab period that it is assigned in. The week it is assigned in can be seen on D2L in Content | Start Here | Class/Lab schedule.